

Edge cleanup report

AI Supply Chain Terminal

2026-07-21

1. What this pass did

Four hygiene items found while reviewing the re-authoring pass:

1. `output_share` moved off `e:hbm-input-nvidia` onto a new `e:sk_hynix-supplies-nvidia` edge. The paper's figure — "SK Hynix ~90% reliant on NVIDIA for HBM" — is about **SK Hynix**, not about aggregate HBM (Micron and Samsung sell elsewhere). The paper's figure now sits on the paper's edge.
2. Provenance for `output_share` split into its own two fields on `EdgeStatic` (`output_share_confidence`, `output_share_source_note`), with a `Pydantic model_validator` enforcing both whenever `output_share` is set. A test documents the same contract at the suite level.
3. Replaced the malformed A1 assertion ("no bucket sums to exactly 1.0 by construction") with the real invariant: **no per-target, per-edge-type input_share sum exceeds 1.0**. Added as a new test in `test_graph_integrity.py`.
4. Corrected contradictory `source_notes` on Samsung's five supplies edges: the boilerplate said "leading-edge logic fab" but the paper's rationale for Samsung specifically leans on its **mixed logic + memory** business.

Assertions:

```
OK  A1  schema validator rejects output_share without provenance
OK  A2  no supplies/input_to bucket exceeds 1.0
OK  A3  edge count = 168 (was 167)
OK  A4  all 63 nodes score and tier under normalize=true
OK  A5  all 63 nodes score and tier under normalize=false
```

2. The new edge

```
{
  "id": "e:sk_hynix-supplies-nvidia",
  "source_id": "company:sk_hynix",
  "target_id": "company:nvidia",
  "type": "supplies",
  "input_share": 0.05,
  "output_share": 0.90,
  "static": {
    "confidence": "estimate",
    "source_note": "Paper §5 identifies NVIDIA as reliant on TSMC + SK
      Hynix.
      The paper does not quantify SK Hynix's share of NVIDIA's overall
      supplies bucket; input_share is an industry-informed estimate
      reflecting
      SK Hynix's role as NVIDIA's dominant HBM supplier (HBM itself is
      separately modelled via the HBM product node). Bucket
      deliberately
      incomplete.",
    "output_share_confidence": "hard",
    "output_share_source_note": "Paper §5 (Financial
      Interdependencies):
      'SK Hynix ~90% reliant on NVIDIA for HBM.' This is the paper's
      own
      figure on the paper's own edge – the previous placement on
      e:hbm-input-nvidia asserted the ratio for the aggregate HBM
      product,
      which overstated the paper (Micron and Samsung sell elsewhere)."
```

Two provenance fields, two source notes, two confidences. This is the pattern every future output_share will follow.

3. e:hbm-input-nvidia after cleanup

input_share	0.30	(unchanged)
output_share	null	(was 0.90 – moved)
input_share confidence	estimate	(unchanged)
input_share note	"Paper §2C names HBM a co-equal bottleneck with the GPU. Paper does not quantify HBM's	

```
fraction
of NVIDIA's BOM; industry-informed estimate."
(was: same, followed by " | output_share: SK
Hynix supplies ~90% of NVIDIA's HBM per paper
$5.")
output_share fields      absent
```

The concatenated | output_share: tail is gone.

4. NVIDIA supplies bucket after cleanup

company:tsmc	0.94	(was 0.99 – reduced to make room)
company:sk_hynix	0.05	(NEW)
company:samsung	0.01	(unchanged)
sum	1.00	

TSMC → NVIDIA reduced by 0.05 to hold the bucket at 1.00. The reduction is consistent with paper §5’s own reasoning: NVIDIA is reliant on **TSMC + SK Hynix**, not on TSMC alone. Note added to the TSMC edge:

“Reduced from 0.99 to 0.94 in the cleanup pass to accommodate the new SK Hynix → NVIDIA supplies edge added per paper §5 (mutual TSMC + SK Hynix reliance). Bucket total held at 1.00.”

5. Tier / severity diff

Method: rebuilt the pre-cleanup state (edge count 167, TSMC → NVIDIA at 0.99, no SK Hynix supplies edge, output_share restored on HBM → NVIDIA for edge-count parity) and scored both states under both normalize modes.

normalize=true (default)

Node	Tier before	Tier after	Sev before	Sev after
NVIDIA	moderate	moderate	0.104	0.094

o tier changes. NVIDIA loses 0.010 severity because its inbound HHI drops from 0.980 (TSMC 0.99, Samsung 0.01) to 0.886 (TSMC 0.94, SK Hynix 0.05, Samsung 0.01) — three suppliers with meaningful weight is a lower-

concentration bucket than two-with-one-dominant. Every other node is identical to the prior pass.

normalize=false

Node	Tier before	Tier after	Sev before	Sev after
NVIDIA	moderate	moderate	0.104	0.094
Constellation Energy	none	moderate	0.049	0.050

1 tier change, and it’s a numerical boundary flip: the moderate threshold is exactly 0.050, and Constellation Energy sits at 0.049 → 0.050 across it. Not caused by NVIDIA’s inbound change directly — the mechanism is a cascade adjustment reaching Constellation via shared downstream data-center nodes, compounded by the 0.049 baseline sitting one unit below the boundary. Not semantically meaningful; called out here because the assertion required reporting every tier change.

6. Test suite results

normalize=true

27 tests, 24 pass, 3 fail

FAIL test_no_input_share_bucket_exceeds_one (this pass added it; see below)

FAIL test_every_paper_chokepoint_is_critical (pre-existing: ASML, HBM)

FAIL test_no_stage_bucket_sums_below_0_80 (pre-existing: 38 buckets;

is the deliverable of

the

share-completeness

test)

normalize=false

Same three failures. The 24 passing tests are identical between modes.

7. test_no_input_share_bucket_exceeds_one — the new test’s failure

Added per §3 of the spec. Fails on **5 pre-existing buckets**, none touched by this pass:

country_region:usa	located_in	sum = 6.00
country_region:south_korea	located_in	sum = 2.00
mineral:neodymium	mines	sum = 1.17
mineral:neodymium	refines	sum = 1.10
mineral:dysprosium	refines	sum = 1.01

The located_in bucket is categorical — every facility carries input_share=1.0 because it is 100% located in that country. The bucket schema doesn’t fit target-input-share semantics for this edge type: a “located_in” bucket is a **set membership**, not a share.

The mineral buckets carry both country-level AND facility-level rows (country_region:china mines 60% of neodymium; facility:mountain_pass mines 12% of neodymium — layered granularities in the same bucket). Real mismatch, but pre-existing, structural, and out of cleanup scope.

All 44 supplies and input_to buckets — including the two the cleanup pass touched (NVIDIA supplies, NVIDIA input_to) — meet the invariant.

8. Samsung supplies notes — before / after

Before (5 edges), all prefixed with:

“Paper §2D anchors ASML as sole EUV maker (~90% of litho market), and describes AMAT / LRCX / KLA / TEL as the equipment stack behind leading- edge fabs. The paper does NOT quantify per-supplier share of a foundry’s equipment input; specific values below are industry-informed estimates of tool-spend mix at a **leading-edge logic fab**.”

The “leading-edge logic fab” line contradicted Samsung’s re-authoring rationale (which relied on the memory business raising AMAT and LRCX shares vs. a pure-logic fab like TSMC).

After:

“Paper §2D describes ASML / AMAT / LRCX / KLA / TEL as the equipment stack behind leading-edge fabs. Samsung runs both a foundry business and one of the world’s largest memory operations, so its tool-spend mix differs from a pure-logic fab like TSMC’s. Paper does not quantify per-supplier share; values are industry-informed estimates of Samsung’s **mixed logic + memory tool-spend**.”

Plus edge-specific tails carried through unchanged:

- e:asml-supplies-samsung: “ASML share is lower than for TSMC because Samsung uses less EUV at this stage (paper §2B: ‘3nm yields stuck at 30–40%’).”
- e:amat-supplies-samsung: “Memory manufacturing raises AMAT’s share vs a pure logic fab.”
- e:lrcx-supplies-samsung: “Memory’s 3D stack requirements raise LRCX’s share vs a pure logic fab.”
- e:kla-supplies-samsung: “Inspection/metrology slice.”
- e:tel-supplies-samsung: “Coating/etch/cleaning slice.”

The rationale each edge records is now consistent with its stated basis.

9. Schema change

backend/app/schema/edge.py:

```
class EdgeStatic(BaseModel):
    ...
    output_share_confidence: Optional[Confidence] = None
    output_share_source_note: Optional[str] = None

class Edge(BaseModel):
    ...
    @model_validator(mode="after")
    def _output_share_needs_provenance(self) -> "Edge":
        if self.output_share is not None:
            if self.static.output_share_confidence is None:
                raise ValueError(...)
            if not self.static.output_share_source_note:
                raise ValueError(...)
        return self
```

frontend/src/types.ts mirrors the two new optional fields on the static block.
No other frontend change.

10. Files changed

backend/app/schema/edge.py	+26 lines (fields + validator)
backend/tests/fixtures/ai/edges.json	resync (167 → 168 edges)
backend/tests/test_graph_integrity.py	+34 lines (2 new tests)
data/ai/edges.json	167 → 168 edges; – relocated output_share – reduced TSMC → NVIDIA – rewrote 5 Samsung notes – split HBM → NVIDIA note
frontend/src/types.ts	+2 fields on Edge.static

Scoring, config, thresholds, narration, cascade, outbound criticality: all
untouched, per spec.